

# Single Page Application & Desenvolvimento Responsivo

**Parte #01** - A Evolução do SPA e do Desenvolvimento Responsivo no Contexto Histórico da Web



# Sistemas Desktop



- Até o início dos anos 2000 as aplicações eram predominantemente executadas em Desktop, desenvolvidas por linguagens populares como **Visual Basic**, **Delphi** e **Java Swing**.
- O principal desafio das aplicações Desktop era a **distribuição e manutenção**. Atualizar dezenas ou centenas de computadores era caro e demorado. Além disso, aplicações cliente-servidor dependiam fortemente da infraestrutura da rede local e do banco de dados.

# WEB 2.0: A onda dos Sistemas WEB no início dos anos 2000

- Linguagens como **PHP**, **ASP**, **JSP** e **ECMAScript / JavaScript** permitiram que Grandes empresas de tecnologia criassem sistemas inteiros para executar no navegador: Webmail, CRM, ERP, Mensageiros, e-Commerce.
- Outras organizações começaram a recriar suas aplicações Desktop para web criando um forte movimento de migração de tecnologias.



# Junto com a novidade, novos problemas! O dilema do MPA - Multi Page Application.

- **Design Fixo:** Os sites tinham larguras fixas. Eles eram projetados exclusivamente para as resoluções padrão dos monitores de computadores da época.
- **A cada clique, o navegador faz uma requisição completa ao servidor / payload:** reprocesso, consumo de recursos desnecessários do servidor e gastos elevados com nuvem (cobrado por consumo).
- **Perda de Estado da Aplicação:** posição do scroll, preenchimento de formulários.
- **Fluidez / Latência e Experiência do Usuário - UX:** branco constante, gargalos e atrasos.

# GMAIL! Se o Google consegue, nós também conseguimos!

- 1º de abril de 2004 o GMAIL é lançado pela Google (Mas ainda não era um SPA puro - o conceito foi melhorado gradativamente).
- Um webmail altamente dinâmico, onde grande parte da interface era atualizada sem recarregar a página usando o objeto **XMLHttpRequest** do JS que permitia uma comunicação assíncrona entre o navegador e o servidor.
- "Se o Google consegue fazer isso, nós também podemos." Outras aplicações interessantes e populares começaram a surgir como *Flickr*, *Facebook*, *Twitter*, *Salesforce*.
- Mas ainda não eram responsivos. “JS” e “CSS” eram mundos separados.

# O Boom do Mobile

- Com o surgimento dos smartphones modernos, acessar a web pelo celular tornou-se frustrante. Os usuários precisavam dar "zoom" constantemente e rolar a tela de forma horizontal para conseguir ler qualquer conteúdo.
- Empresas com Apple e Google começaram a investir em mecanismos para melhorar a experiência do usuário, baseado estudos de IHC.
- Em 2007 é o iPhone. O navegador Safari móvel, que permitia renderizar sites completos de desktop na palma da mão.



# A solução! O AJAX

- Em 2005 o termo AJAX foi criado para descrever um conjunto de técnicas, baseadas em JS, capazes de atualizar partes da página sem realizar um recarregamento completo.
- Baseado em JavaScript.
- Foi a primeira aplicação de grande impacto a popularizar essa abordagem. Tudo era carregado dinamicamente.
- O conceito de SPA (Single Page Application) começa a surgir e gradativamente sendo melhorado.

# A Ruptura do iPhone

O enorme sucesso de vendas do iPhone a partir de 2007 gerou uma explosão no tráfego mobile.

Os desenvolvedores entraram em crise, pois era inviável criar um site diferente para cada novo tamanho de tela que chegava ao mercado.

Foi exatamente essa dor de cabeça causada pelo iPhone que forçou a criação e a rápida adoção do **CSS3** com **Media Queries** e da **filosofia responsiva em 2010**.

# A Evolução! O JQuery

- em 2006 o **jQuery** surge como uma biblioteca JS que prometia “Escreva menos e construa mais”.
- Ele simplificava o JS, reduzia consideravelmente o código, mas...
- ... projetos grandes se tornavam caóticos e sofriam com conflitos de navegadores; Resolver diferenças entre navegadores era uma das maiores dores do desenvolvimento web naquela época.

# A Novo Padrão de Indústria de Software: Mobile First

- **A Origem:** O termo foi cunhado pelo designer Luke Wroblewski em 2009, num momento em que o uso de smartphones começava a ganhar muita força, e foi popularizado mundialmente com o lançamento de seu livro homônimo em 2011.
- O Conceito (Inversão de Lógica): Em vez de criar um site complexo para monitores de computador e depois tentar cortar ou "espremer" elementos para caber no celular, o design começa obrigatoriamente pela tela menor e mais restrita.
- **Aprimoramento Progressivo:** Ao focar no celular primeiro, você prioriza o conteúdo e as ações que são absolutamente essenciais. A partir daí, à medida que a tela aumenta (para tablets e desktops), novas funcionalidades e elementos visuais mais complexos vão sendo adicionados progressivamente.

# O Twitter e o Laboratório das SPAs (2010 - 2012)

- Ainda em 2010, o Twitter lançou o "#NewTwitter". A empresa reescreveu sua arquitetura quase inteiramente em JS no lado do cliente, com o objetivo de fazer a aplicação se comportar de maneira rápida e contínua, sem recarregamentos de tela e 100% responsivo.
- A primeira a chamar a atenção pela técnica *hashchange*.
- **Problema:** O carregamento inicial era muito lento e forçou uma alteração; enderizar o carregamento inicial no servidor (Server-Side Rendering) para entregar o conteúdo mais rápido, deixando o JavaScript assumir a navegação apenas depois que a página já estava visível.

# A Revolução de 2010! Os primeiros frameworks e a componentização

- Backbone.js, Knockout.js e Ember.js prometiam mais organização em relação do jQuery.
- Introduziram conceitos como MVC, Data Binding e Roteamento.
- O divisor de águas entre aplicações tradicionais para aplicações SPAs modernas.
- Componentes como botões, menus, inputs e blocos como header e footer já eram pensando em como se comportar (resposta) ao tamanho de tela do dispositivo. Tecnologias como **CSS-in-JS**.

# O Precursor AngularJS

- Em 2010 a Google apresenta o AngularJS (não confundir com o Angular, que é baseado em TypeScript).
- Inspirou diversos outros framework e até mesmo significativa melhorias no jQuery.
- JQuery começa a perder popularidade para aplicações comerciais.
- Melhorou os conceitos de controladores, templates e rotas.

# A Ascensão das Bibliotecas de UI (A Comoditização)

- **Bootstrap (2011):** Originalmente lançado em agosto de 2011 pelos desenvolvedores do Twitter. Ele não nasceu focado em componentes de JS, mas foi crucial para popularizar o sistema de grids responsivos na web.
- **Material UI / MUI (2014):** Iniciou sua história em 2014 como uma implementação em React das diretrizes visuais do Material Design. É quase tão antigo quanto o próprio ecossistema do React.
- **Tailwind CSS (2017):** Lançado inicialmente em novembro de 2017, revolucionou o mercado moderno com sua abordagem de classes utilitárias (utility-first).



# O Inovador REACT!

- Surgiu em 2013 e apresenta outra abordagem: Não manipulava diretamente o DOM; utilizava o Virtual DOM para carregar apenas as alterações necessárias antes de atualizar o DOM real. Ex.: Não carrega o menu mobile enquanto o usuário não tocar nele.
- Aplicações grandes começaram a ficar mais “leves”.
- A partir destes novo conceito, outros frameworks foram criados como Vue.js, Angular, Svelte.

# ES6 (ECMAScript 2015): A maior atualização da história e no momento certo!

- O mercado precisava de tecnologias confiáveis para aproveitar a popularização da ADSL, dispositivos móveis (Pockets PCs e smartphones),
- Suporte à classes,
- adição do ***let*** e ***const***,
- *arrow functions*,
- desestruturação,
- módulos nativos e APIs de navegador como *localStorage*,
- **jQuery e similares não é mais necessário!**
- **A explosão das SPAs com *Bundlers* (*Vite*, *Webpack*).**
- **IE morre e uma nova era de navegadores modernos se inicia.**

# O TypeScript de 2018:

- Criado pela Microsoft,
- Adiciona mais funcionalidades ao JS, tipagem estática, depuração de erros em tempo de desenvolvimento (no editor de código), muito antes de ir para produção e quebrar no navegador do usuário.
- Mais segurança, menos falhas, menos código,
- As SPAs deixaram de ser pequenos scripts e passaram a ser sistemas empresariais gigantescos,
- Gradativamente o TS se torna um padrão no desenvolvimento de frontend.
- Fim do *undefined is not a function*.

# Frameworks

- Angular
- React
- Vue.js
- Svelte (A nova geração)

# Exemplo Completo no GitHub

